

CLASS 04 / SEP 15 / U04

ARRAYS AND LISTS

PART ONE

P2 BEGINS — ONE COMPONENT, MANY POSITIONS

P1a DUE

P2a ISSUED TODAY

AN ARRAY IS A LIST OF POSITIONS

Not a command that duplicates things. A list you compute, then place geometry at.

INDEX → POSITION

$i = 0 \rightarrow p = \text{start}$

$i = 1 \rightarrow p = \text{start} + \text{step}$

$i = 2 \rightarrow p = \text{start} + 2 \times \text{step}$

$p[i] = \text{start} + i \times \text{step}$

THREE THINGS TO READ

THE LIST — what is in the collection.

THE COUNT — how many there are.

THE INDEX — where each one sits in the order.

The index is the only new idea. It is a counter that tells each copy how far along it is — and therefore how different it should be.

Your pyramid is the component. It does not change today — only where it goes.

THE REPEAT ZONE

The loop, made visible. Repeat Input, a body, Repeat Output.

WHAT IT IS

Repeat Input → **body** → **Repeat Output**

Everything between the two runs once per iteration. Whatever you hand to Repeat Input comes back changed, once for every step of the count.

READ IT SMALL FIRST

Set the count to 3 and look at the spreadsheet.

Then 5. Then 20.

A collection you cannot inspect is a collection you cannot debug.
There is no prize for a large count.

BUILD IT AT A COUNT YOU CAN COUNT. SCALE UP ONLY ONCE IT IS RIGHT.

COMPARE + SWITCH IS AN IF STATEMENT

Test a value, then act on the answer. Two nodes, one idea.

THE PATTERN

```
Compare  index > 4      → True / False
Switch   False → a
          True  → b
```

Compare asks the question. Switch chooses between two answers.
Everything conditional in this course is those two nodes.

WHAT IT BUYS YOU

Move only a range of indices.

Cull every third element.

Give the first row a different height.

The field stops being uniform the moment a test decides something.

COMPARE THE RESULT AT TWO DIFFERENT COUNTS. IF IT ONLY WORKS AT ONE, IT IS NOT A RULE.

NEST IT AND YOU HAVE A GRID

Two counters instead of one. A list of lists.

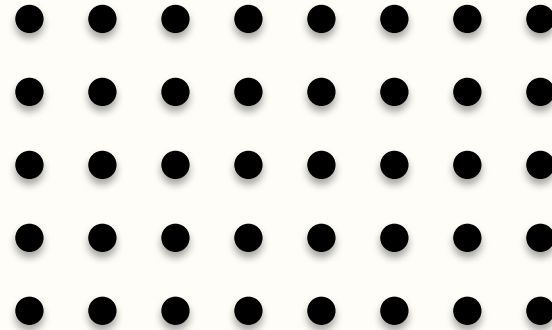
ONE COUNTER



`p[i] = start + i × step`

`count = 8`

TWO COUNTERS



`p[i][j] = start + (i × dx, j × dy)` `count = 8 × 5`
`= 40`

PREDICT THE TOTAL BEFORE YOU LOOK. IF THE NUMBER SURPRISES YOU, THE NESTING IS WRONG.

THE SAME THING, WRITTEN OUT

Everything in U04 is in these nine lines — the nested loop, the stagger, the conditional, the sine.

```
points = []

for j in range(rows):                # nested loop = grid
    offset = spacing / 2 if j % 2 else 0  # odd/even stagger
    for i in range(cols):
        z = amplitude * sin(i * frequency + phase)
        if 3 <= i <= 5:                # Compare + Switch
            z = z + lift
        points.append([i * spacing + offset, j * row_gap, z])
```

WHAT IT MAPS ONTO

list	modulo
index	sine
count	amplitude
loop	frequency
nested loop	phase
conditional	

THE INDENTATION IS THE NESTING. THAT IS WHY A GRID IS A LOOP INSIDE A LOOP.

ISSUED TODAY

P2a — ARRAYS AND CONDITIONAL CONTROL

DUE WEEK 8 — Oct 13 / 7%

Use the geometry project as a component. First design its arrangement, then vary it through external measurements. This milestone is the arrangement half: four ways of placing the same component, three of them developed, with the repetition and selection logic built from basic nodes.

BUILD

FOUR ARRANGEMENTS

Linear, nested grid, hexagonal, radial — the grid is required.

CONTROL

ONE CONDITIONAL

An index range displaced, using Compare and Switch.

VARY

ONE SINE

Amplitude, frequency and phase, named and demonstrated.

REQUIREMENTS — P2a

Visual quality 35% · computational understanding 25% · technical execution 20% · experimentation 10% · requirements and identity 10%.

01

Practice linear, nested-grid, hexagonal and radial arrays; develop three comparisons on up to two sheets, including the nested grid.

02

Keep the component constant while comparing arrangements. Expose count and spacing or radius and explain index order.

03

Show one conditional index-range displacement and one sine-driven variation; identify amplitude, frequency and phase.

04

Build the repetition and selection logic from basic nodes. Show a small inspectable count before scaling up; no minimum of 1,000 elements is imposed.

NOW — YOUR FIRST ARRAY

Rest of the session: a linear array, a conditional, and a grid you can predict.

✓

Your pyramid instanced along a linear array, count and spacing exposed

✓

The spreadsheet open — list, count and index all readable at count = 5

✓

Compare and Switch changing one range of indices

✓

The array nested into a grid, and you predicted the total before looking

✓

Index order explained out loud: which way does i run, and what happens if you swap it

A COUNT YOU CAN COUNT. NOT A THOUSAND OF ANYTHING, YET.

BEFORE NEXT CLASS

CONTINUE

U04 — the same unit runs into next week. Hexagonal and radial arrangements, conditional displacement and sine control.

BUILD

P2a is due at the midterm. Start the comparisons now — the sheet is easier when the graphs already exist.

BRING

Your grid, working, at a small count. And P1b — the pyramid sheet is due next class.

CHECK

P1c: print-ready geometry, physical units and orientation get reviewed next week before anything goes in the queue.